Information Security, Assignment 1

RekamKu: An Encrypted Personal Health-Record Vault

Group 10 Kartika Nana Naulita (5025241021), Safa Mashita (5025241022), Devina Balqis Aurora (5025241034), Via Hana Nurma Putri (5025241048)

Repository: https://github.com/mashitaaa/RekamKu | Website: https://rekamku.pythonanywhere.com |
AI Usage: https://github.com/mashitaaa/RekamKu/blob/main/docs/AI_USAGE.md

1. Scenario and why the application is useful

Patients with chronic conditions such as epilepsy, diabetes or hypertension are often treated in several places: a puskesmas, a referral hospital, a private laboratory. Their lab results, discharge summaries, the KTP and BPJS scans requested at registration, and the medical videos doctors ask for (for example, a seizure filmed by a family member) end up on paper or in a phone gallery. In that state, a lost phone, an automatic cloud backup or a message sent to the wrong chat group is enough to expose a diagnosis.

RekamKu gives these patients, or the carers who manage their accounts, one place for these records: a health profile, identity scans, PDF/DOCX/XLSX medical documents and videos, which they can open or download at the next consultation. Because the records belong to the patient, the requirement that only the authenticated user gets their own data back, and gets it back intact, is exactly the behaviour this setting needs. Law No. 17/2023 on Health gives patients the right to access the information in their medical record (Art. 276(e)); RekamKu is where they can keep it.

2. Which data is private (Law No. 27/2022 on Personal Data Protection, UU PDP; GDPR)

Data	Class	Basis and consequence
Diagnoses, allergies, medication, blood type, BPJS number, medical documents, medical videos	Specific	Health data, including use of health services (UU PDP Art. 4(2)(a)); special category under GDPR Art. 9(1).
KTP and BPJS card scans	Specific (handled as)	Name, NIK and address are general data. The face photo is potentially biometric: the elucidation of Art. 4(2)(b) lists facial images as an example when they allow unique identification, whereas the GDPR treats a photo as biometric only when it is processed by specific technical means (Recital 51), which RekamKu does not do. The BPJS card reveals use of health services (Art. 4(2)(a)), so the whole folder is treated as specific.
Name, NIK, date of birth, sex, address, phone, emergency contact (a third party)	General	Art. 4(3)(a)(b)(f). Combined with a medical record they identify the patient together with the illness.
Random IDs, username HMAC, salts and KDF parameters, log public key, ciphertext size	Pseudonymous technical data	Still personal data, because each value is linked to one account (GDPR Recital 26). They stay readable only because sign-in needs them before any user key exists, and none of them reveals content.

All personal data, general and specific, is encrypted, because general data stored next to health data reveals just as much. The classification is still used: it determines the legal basis, the labels in the UI and the /privacy page. RekamKu relies on explicit consent, one of the six bases listed in Art. 20(2) (GDPR Art. 9(2)(a)), collected through a mandatory checkbox at registration. The data-subject rights are implemented as features: download (Art. 7), profile editing (Art. 6), deletion of a file or the whole account (Art. 8) and an activity log (Art. 16(2)(h)).

3. Design decisions and their justification

Decision	Choice	Reason
Applying AES, DES and RC4 to everything	Cascade: plaintext → DES-CBC → RC4 → AES-256-GCM, each layer with its own HKDF-derived key	Every byte passes through all three ciphers. Keeping one copy per cipher would let an attacker target the weakest copy (56-bit DES) and would triple storage; assigning one cipher per file would leave some records protected by RC4 alone.
Layer order	AES-GCM outermost	The GCM tag is checked before RC4 or DES touch the data (encrypt-then-MAC). Modified ciphertext is therefore rejected before any plaintext is produced, which also rules out a DES-CBC padding oracle.
Modes of operation	AES: GCM, nonce = chunk index. DES: CBC, random IV per chunk. RC4: stream cipher	ECB is excluded by the assignment, and for good reason: equal blocks give equal ciphertext, which would leak patterns in scans and repeated form fields. GCM provides confidentiality and integrity in one pass, and using the chunk index as nonce is safe because every object has its own AES key (HKDF, random 24-byte salt). For DES, CBC was preferred over CTR, since a repeated CTR nonce repeats the keystream, the same weakness RC4 has. Sweet32 would need ~2^32 blocks (32 GB) under one key, while a file is at most 200 MB.
RC4	RC4-drop[3072], per-chunk key = HMAC(k, index)	Dropping the first 3072 bytes discards the most biased part of the keystream. A fresh key per chunk means a keystream is never reused; reuse would leak the XOR of two plaintexts, as E6 shows.
File format	1 MiB chunks; AAD = H(header‖owner‖id) ‖ index ‖ last flag	Chunking keeps server memory constant and makes videos seekable (HTTP Range). Each chunk is verified before its bytes are sent; if a later chunk fails, the transfer stops, so the client may get an incomplete download but never unverified data. Through the AAD, modification, reordering, truncation and swapping files between users are all detected.
Key handling	Envelope: password → scrypt+pepper → KEK → UMK → per-file FEK	Changing the password only re-wraps the UMK. Deleting a file removes its wrapped FEK and blob from the live system, but a database backup taken earlier still holds the wrapped FEK. The pepper from the server key stops offline guessing on a stolen database only if that key is stored elsewhere (REKAMKU_SERVER_KEY); by default it lies on the same disk (instance/server.key). A 100-bit recovery code wraps the same UMK, so a forgotten password does not cost the patient their records.
Password KDF	scrypt N=2^17, r=8, p=1	scrypt is memory-hard (128 MiB per guess). It measured 589 ms per sign-in (E7), an acceptable wait; N=2^15 takes 163 ms and would make guessing ≈3.6× cheaper.

Where encryption runs	Server side, keys derived from the user's password	Browser-side encryption was rejected because WebCrypto has no DES or RC4, and the server has to check file types and serve Range requests. This protects against a stolen database, disk or backup, a curious storage admin and files modified at rest, but not against a compromised running server (Section 5).
Metadata and activity log	File metadata inside the encrypted envelope and the log entries sealed to the user's X25519 key	File names, categories, sizes and dates are encrypted along with the content. Failed sign-ins have to be logged while the vault is locked, so each entry is encrypted with the same cascade under a one-time X25519 key. only the owner's private key, wrapped by the UMK, can open it. A test checks every database column.
Architecture, storage and access	Flask web app in one process; SQLite + randomly named blobs; session, CSRF and owner check on every query	One process is required because session keys live in RAM. SQLite is enough for a personal vault and adds no separate database server to secure, since every personal column is encrypted anyway. Uploads arrive as a raw streamed body, because Werkzeug writes multipart files over 500 KB to /tmp in the clear, and files are written atomically. The cookie holds only a random session ID, idle sessions expire after 15 minutes and sign-in is limited to five attempts per 15 minutes. The AAD additionally binds each file to its owner.

4. Analysis: what we measured, why it is the right analysis here, and what it means

We compared AES, DES and RC4 on what matters to a patient and to a small server: the time to encrypt and decrypt the files patients actually keep (from a small spreadsheet to a three-minute video), each cipher's share of the cascade time, the server memory needed for large videos, whether a silent change to a stored lab value is caught, and the extra storage that encryption adds. These measurements follow the risks of the scenario. A patient waits at every upload and at the next consultation, many patients share one small server, and a wrong lab value can lead to wrong treatment. A speed-only benchmark, or a comparison of key lengths, would miss these points.

To keep disk speed out of the results, the data was loaded into memory first. Each test had two warm-up runs followed by 40 repetitions for small files and 12 and 7 for the two videos, with the cipher order shuffled every round and the garbage collector switched off; we report the median and the IQR. All tests ran on an Intel Core i7-13620H laptop (15.7 GB RAM, NVMe SSD, Windows 11, AES-NI available) with Python 3.13.3 and pyca/cryptography 50.0.2. The suite was run twice on the final code, and the results stayed close. AES-GCM varied the most, because its times are so short that timer noise weighs more. The gaps between the ciphers are far larger than this variation (for the 90 MB video, DES is about 70 times slower than AES-GCM), so the conclusions do not depend on it. Figure 1 and all numbers come from the first run; absolute times will differ on other machines. Scripts and raw CSV files are in bench/.

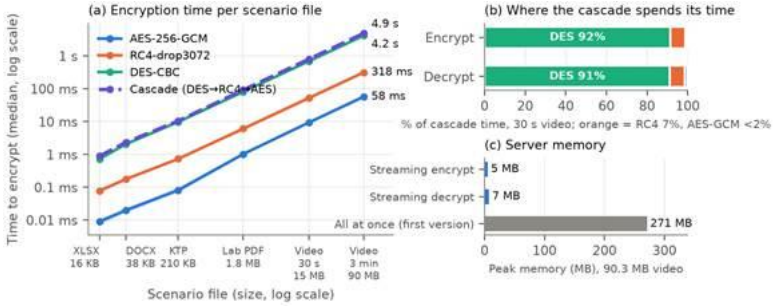


Figure 1. (a) Median encryption time (band = IQR); decryption is similar. (b) Share of each layer in the cascade, 30 s video. (c) Peak Python heap (tracemalloc) while encrypting the 90 MB video; "all at once" is a whole-file baseline re-implemented in the benchmark script.

- DES is the expensive part; AES-GCM is almost free. AES-256-GCM runs at gigabytes per second, RC4 at hundreds of megabytes per second and DES at only about 20 MB/s. Inside the cascade, DES takes over 90% of the time, RC4 a small share and AES-GCM about 1%. Part of the DES cost comes from the library: pyca only offers DES as 3DES-EDE with three identical keys, which gives the same output as single DES (it passes the FIPS 81 test vector) but needs three passes. Real single DES would be up to about three times faster and would still take most of the cascade time.
- Patients barely notice the cost. Encrypting a KTP scan takes about 10 ms and a 1.8 MB lab PDF about a tenth of a second; a full upload through the app (Flask test client, no network or TLS) adds only a little on top. A three-minute video takes about five seconds to encrypt and about the same to decrypt. Over an assumed 10 Mbit/s uplink, uploading that video takes more than a minute anyway, so encryption adds only about 7% to the wait. Because of chunking, jumping to the middle of a video takes roughly a tenth of a second for both the 15 MB and the 90 MB file, whereas decrypting the whole 90 MB file takes several seconds.
- Server capacity is limited by DES, not AES. One core is busy for about five seconds per three-minute video, almost all of it in DES; with AES-GCM alone, the cipher time would be well under a tenth of a second. Streaming keeps memory use at a few megabytes, while the whole-file baseline needs 271 MB, three times the size of the video.
- Integrity in practice (E6). An attacker who can write to storage but has no key changes “Fasting glucose: 95” into “35”. With RC4 alone the result decrypts to exactly “35 mg/dL” and nothing flags it; with DES-CBC alone it also passes, with only the preceding block garbled. AES-GCM and the cascade both reject it, and this is the main reason AES-GCM is in the design. Reusing an RC4 key also leaked “Diagnosis: epilepsy” from the XOR of two ciphertexts, which is why RC4 keys are per chunk. Storage overhead is negligible: under half a percent for the smallest file and practically nothing for the video.
- Conclusion: Confidentiality and integrity rest on AES-256-GCM, which is also the cheapest layer. DES (56-bit key, broken by the EFF in 56 hours in 1998) and RC4 (keystream biases, prohibited in TLS by RFC 7465) are included only because the assignment requires them. Because they sit inside the GCM layer, the design does not depend on their strength, and we do not recommend them for production. Outside this assignment we would use AES-256-GCM alone, which would also remove most of the measured cost.

5. Limitations

The server sees plaintext while a session is active, because encryption happens on the server rather than end-to-end; a compromised running server therefore remains a risk. Session keys in RAM restrict the app to one process, and Python cannot guarantee that memory is wiped. Ciphertext sizes are visible. Since the AAD carries no version number, someone with write access to storage could restore an older valid ciphertext of the same record (rollback) without detection. Consent is required at registration but is not stored with a timestamp, although Art. 22(1) requires written or recorded consent. Losing both the password and the recovery code makes the data unrecoverable, a trade-off accepted on purpose. All 49 automated tests pass (official vectors, IDOR, bit flips, truncation, blob swapping, CSRF, no plaintext in the database or on disk).